

Enhancing the security in IoT and IIoT networks: An intrusion detection scheme leveraging deep transfer learning

Basharat Ahmad^{a,b}, Zhaoliang Wu^{a,b,*}, Yongfeng Huang^{a,b}, Sadaqat Ur Rehman^c

^a Tsinghua National Research Center for Information Science and Technology, Tsinghua University, Beijing, 100084, China

^b Department of Electronic Engineering, Tsinghua University, Beijing, 100084, China

^c School of Science, Engineering and Environment, University of Salford, Manchester, M5 4WT, United Kingdom

ARTICLE INFO

Keywords:

Internet of things (IoT)
Industrial internet of things (IIoT)
Intrusion detection system (IDS)
Deep transfer learning (DTL)
Cybersecurity
Lightweight IDS

ABSTRACT

The Internet of Things (IoT) networks, which are defined by their interconnected devices and data streams are an expanding attack surface for cyber adversaries. Industrial Internet of Things (IIoT) is a subset of IoT and has significant importance in-terms of security. Robust intrusion detection systems (IDS) are essential for protecting these critical infrastructures. Our research suggests a novel approach to the detection of anomalies in IoT and IIoT networks that leverages the capabilities of deep transfer learning. Our methodology begins with the EdgelloT dataset, which serves as the basis for our data analysis. We convert the data into an appropriate image format to enable Convolutional Neural Network (CNN)-based processing. The hyper-parameters of individual machine learning models are subsequently optimized using a Random Search algorithm. This optimization phase optimizes the performance of each model by modifying the hyper-parameters that are unique to the learning algorithms. The performance of each model is meticulously assessed subsequent to hyper-parameter optimization. The top-performing models are subsequently, strategically selected and combined using the ensemble technique. The IDS scheme's overall detection accuracy and generalizability are improved by the integration of strengths from multiple models. The proposed scheme demonstrates significant effectiveness in identifying a broad spectrum of attacks, encompassing a total of 14 distinct attack types. This comprehensive detection capability contributes to a more secure and resilient IoT ecosystem. Furthermore, application of quantization to our best models reduces resource utilization significantly without compromising accuracy.

1. Introduction

A technological transformation is underway in the industrial landscape as advanced digital technologies and the Internet of Things (IoT) combine. This convergence heralds in the Fourth Industrial Revolution (Industry 4.0), characterized by a highly networked world in which the digital and physical realms coexist together. The emergence of Industry 4.0 and the IoT, with particular reference to the Industrial Internet of Things (IIoT) are the focus of researchers. It underlines how important robust cybersecurity measures are in this quickly changing technology landscape [1].

The IoT is a network consisting of networked physical devices that are equipped with sensors, software, and other technologies. These devices are capable of collecting and exchanging data. These technologies, which include wearable and smart appliances, establish communication with one other and external systems, allowing the development of intelligent surroundings and automated operations [2]. The IIoT is a subset of the broader IoT that is especially dedicated to industrial

applications [3]. IIoT devices, such as sensors installed on factory machinery or embedded chips in power grids, produce real-time data that enables enterprises to enhance operational efficiency, optimize performance, and implement predictive maintenance [2].

Industry 4.0 expands on the groundwork established by the IoT and IIoT, introducing a new era of advanced manufacturing. The driving force behind this transformation lies in the progress made in artificial intelligence, machine learning, big data analytics, and cloud computing. Industry 4.0 utilizes advanced technology to create cyber-physical systems that facilitate intelligent decision-making, autonomous operations, and real-time communication along the whole value chain [4]. Industry 4.0 offers substantial economic and industrial advantages, but its dependence on networked devices and large volumes of data necessitates strong cybersecurity solutions.

The inter-connectivity of Industry 4.0 makes vital infrastructure and sensitive industrial data vulnerable to several cyber-threats. Malicious

* Corresponding author.

E-mail addresses: bash18@mails.tsinghua.edu.cn (B. Ahmad), wuzl@mail.tsinghua.edu.cn (Z. Wu), yfhuang@mail.tsinghua.edu.cn (Y. Huang), srehman15@salford.ac.uk (S.U. Rehman).

<https://doi.org/10.1016/j.knosys.2024.112614>

Received 20 June 2024; Received in revised form 14 August 2024; Accepted 6 October 2024

Available online 11 October 2024

0950-7051/© 2024 Elsevier B.V. All rights reserved, including those for text and data mining, AI training, and similar technologies.

individuals can take advantage of weaknesses in interconnected devices to disrupt operations, pilfer intellectual property, or inflict physical harm. Cyber-attacks in an Industry 4.0 setting can have significant repercussions, including halting production and causing environmental catastrophes.

Thus, safeguarding the digital framework of Industry 4.0 requires a comprehensive and multifaceted strategy. This involves the implementation of strong security procedures, the encryption of sensitive data, the utilization of modern technology for detecting and preventing threats, and the promotion of a cybersecurity-conscious culture within businesses. Industries may fully harness the benefits of the Fourth Industrial Revolution and minimize the related risks by giving priority to cybersecurity.

The proliferation of the IoT and IIoT within Industry 4.0 fosters intelligent industrial operations but exposes them to cyberattacks due to their interconnected nature. Intrusion Detection Systems (IDSs) are paramount for shielding IoT and IIoT networks. Machine learning (ML) and deep learning (DL) techniques are integrated into IDSs to a greater extent. However, limitations like restricted labeled data and the evolving landscape of cyberattacks hinder their efficacy. This paper proposes a novel and robust IDS scheme that leverages Deep Transfer Learning to address these challenges.

Exploring the knowledge gained from models, that have already been trained, to enhance the efficacy of traditional deep learning models, DTL seems as an intriguing method. It can take advantage of expertise from a related field with abundant top-notch data to solve the lack of identifiable information for a particular network. Moreover, DTL exhibits the capability to share information even when the target and source network data possess significant dissimilarities.

This study has presented a fundamental comprehension of IoT, IIoT, and Industry 4.0, highlighting the crucial significance of cybersecurity in this interconnected future. This paper presents a highly effective and resilient IDS built upon DTL. The proposed approach leverages nine convolutional neural network (CNN) models that have already been trained, such as Xception, ResNet50, InceptionV3, EfficientNetB6, EfficientNetV2M, VGG19, MobileNet and InceptionResNetV2. The optimal performance of a CNN architecture hinges on the selection of appropriate hyper-parameters. The described approach employs a random search algorithm to optimize and train these architectures. Following training, the top three performing models are incorporated using the bagging ensemble method. The efficacy of the suggested scheme is meticulously evaluated using the state of the art Edge-IIoTset dataset, specifically designed for IoT and IIoT security.

Now we present some of the key contributions of our research. The Edge-IIoTset dataset undergoes preprocessing, meticulously eliminating redundant feature columns. We also exclude some features which do not have any significant role in predictions. Following this, the complete dataset, after preprocessing, is converted into image format, facilitating the effortless identification of diverse cyberattack patterns.

Our study introduces a novel intrusion detection system for IoT and IIoT networks that leverages deep transfer learning (DTL) and image-based representation. A key novelty lies in the transformation of IoT and IIoT devices' data into image format, enabling the effective application of powerful convolutional neural networks (CNNs) for anomaly detection. This innovative representation significantly enhances feature extraction and pattern recognition capabilities. To enhance robustness and accuracy, we make use of an ensemble approach combining multiple pre-trained CNN architectures, fine-tuned with optimized hyper-parameters. Furthermore, our work addresses the challenge of resource constraints in IoT environments by employing model quantization to reduce computational overhead without compromising performance. Through extensive experimentation, we demonstrate the superiority of our proposed method in detecting various cyberattacks compared to existing approaches.

In summary, our work makes the following key contributions:

- **Image-based representation of IoT data:** A state-of-the-art dataset 'Edge-IIoTset' is pre-processed and then transformed into image dataset to facilitate CNN-based processing for identification of diverse cyberattack patterns.
- **Hyper-parameter optimization:** We use hyper-parameter optimization to get the optimized hyper-parameters for training base learning models. We follow the random search algorithm for finding the optimal hyper parameters.
- **Ensemble of pre-trained models:** We use ensemble learning to combine the results of three best performing models. It helps in minimizing variance and enhancing model's accuracy.
- **Model quantization for resource efficiency:** We quantize the individual base learning models in order to get a lightweight model.

The remaining sections of the paper are organized in the following manner: Section 2 offers a brief overview of the most recent progress in IDSs for IoT and IIoT networks that are based on transfer learning (TL). Section 3 presents the suggested scheme, providing a comprehensive explanation of the dataset used, the preprocessing stages, the data transformation, discussion on DTL and used CNN models, the optimization of hyper-parameters, the implementation of ensemble learning, and the quantization process. Section 4 describes the implementation setup and metrics used in the study for performance evaluation. Section 5 covers assessment of performance, analysis of results, and a comparison with existing frameworks. Section 6 demonstrates the generalization capabilities of the proposed scheme by evaluating it on a different dataset. Section 7 serves as the final part of the study, providing a concise overview of the main discoveries and the potential consequences of the research.

2. Related work

Transfer learning (TL) is being utilized more frequently as a framework for the development of sophisticated IDSs that are specifically tailored for applications that involve IoT/IIoT. We present a selection of the most innovative and advanced research projects in the field of cybersecurity that have been conducted using TL.

The accuracy, efficacy, and robustness of conventional IDSs are increasingly being challenged. In order to address these concerns, Li et al. implemented an IDS for the Internet of Vehicles (IoV) that was based on TL. A local update mechanism was developed to utilize pseudo-labels obtained from unlabeled data during fresh attacks through the use of pre-classifiers. These pseudo-labeled data are subsequently utilized in a multitude of TL cycles. The results of their testing indicated that the TL-based technique obtained an accuracy of approximately 92% [5].

Similarly, Mehedi et al. established a malware detection system for IoT networks by employing DTL. Their approach is evaluated using actual data and employs a highly effective attribute selection procedure and a dependable DTL-based ResNet architecture. Extensive experiments conducted on the ToN_IoT dataset [6] validated the model's efficacy.

Masum et al. conducted a study on the application of TL to the detection of novel intrusion attempts. Additionally, their methodology, TL for Neural Intrusion Detection (TL-NID), employs a two-step process. Initially, they employ a pre-trained VGG-16 model that was originally developed for the ImageNet image recognition task. This pre-trained model efficiently extracts significant elements from the intrusion data. The second phase involves the processing of the retrieved features using a deep neural network (DNN) with a predetermined architecture. The basic structure of a deep neural network (DNN) is as follows: an initial layer for input, two hidden layers with 64 and 8 neurons, and an output layer. Their TL-NID solution achieved a detection accuracy of 89.30% for these distinctive attack types when tested on the NSL-KDD dataset, particularly the KDDTest+ subset that concentrates on novel incursions [7].

Additionally, Mehedi et al. implemented a Decision Tree Learning (DTL) methodology to develop an IDS for In-Vehicle Networks (IVN). The development of an attribute selection technique that can identify anomalous messages is a significant accomplishment of this study. This method enables the precise identification of both benign and malignant behaviors through the utilization of a DTL-based LeNet model. The accuracy rate of 98.1% was achieved by evaluating the efficacy of this technique using a custom synthetic dataset [8].

In a separate investigation, Gou et al. introduced a distributed transfer learning method for IoT intrusion detection. Their approach incorporates transfer learning into distributed network boosting algorithms, thereby enhancing the attack learning process despite initial subpar performance. The evaluation, which employed the KDD Cup 99 dataset, demonstrated a 97.3% accuracy in the detection of attacks [9].

The advancements in DTL and its significant accomplishments in other fields have facilitated the development of robust cybersecurity solutions for IoT networks. A system for detecting network intrusions by analyzing raw network traffic was introduced by Bierbrauer et al. which is based on TL. In order to detect attacks, this system utilized the UNSW-NB15 and CICIDS2017 datasets to train a one-dimensional CNN in conjunction with a preserved random forest model, achieving an accuracy of 96.89% [10].

A privacy-preserving transfer learning-based IDS for multiple sources was developed by Xu et al. The Paillier homomorphic encryption strategy was employed by their methodology to cryptographically safeguard models that were derived from a variety of sources. After the models were encrypted, a multisource transfer learning-based IDS was constructed using XGBoost. This system achieved an accuracy of 93.01% in the transmission of encrypted models from various source domains to the destination domain [11].

Singh et al. implemented a DTL-based model in an additional investigation that was intended to detect darknet network activity by analyzing network traffic statistics. In order to improve precision, the researchers transformed temporal characteristics into visual representations, specifically color images. These images were subsequently input into a pre-existing algorithm that had been trained to extract data efficiently. The researchers determined that VGG19 was the most effective model, exhibiting the highest accuracy of 96% in detecting attacks, after utilizing ten pre-trained models and three baseline classifiers [12].

Abosata et al. introduced a model known as FT-CID in their study, which stands for federated transfer learning-assisted customized distributed IDS. This model was specifically developed to identify intrusions within the context of the IoT. This methodology comprises three critical phases: the acquisition of datasets, the training of FTL-enhanced edge IDS, and the detection of intrusions. The FT-CID model is initially created by the central server using a predetermined configuration. It subsequently generates a local model by analyzing the unique characteristics of a variety of low-power and distorted networks, with a particular emphasis on RPL-based IoT networks. Subsequently, edge IDSs were trained using a combination of worldwide and local parameters, which were updated through transfer learning. The FT-CID model demonstrated robust RPL security by effectively incorporating both local and global elements, which led to an exceptional attack identification rate of 85.52% [13].

The issue of limited labeled data available for training deep learning models was effectively addressed by Rodriguez et al. who proposed a novel IDS that was specifically designed for 5G IoT networks. This framework is explicitly engineered to be highly efficient in 5G IoT scenarios, which frequently involve datasets that are imbalanced and have limited labels. The transfer learning model that is the foundation of the system is based on CNNs. The objective of the proposed approach is to optimize the CNN model by employing transfer learning techniques in order to accurately identify zero-day threats in IoT contexts. The framework demonstrated exceptional results, achieving an exhaustive accuracy of 97.89% during the evaluation of the UNSW-NB15 dataset.

Consequently, it underscored its efficacy in identifying a variety of previously unknown attacks [14].

Yan et al. developed TL-CNN-IDS, a malware detection system that employs transfer and ensemble learning to address the challenges of insufficient and imbalanced training samples in real network environments. The method involves preprocessing the dataset using information gain and fast correlation-based filter (IG-FCBF) feature engineering and converting it to an image format that is suitable for CNN input. Three CNN models — VGG16, Inception, and Xception — are optimized using the Tree-Structured Parzen Estimator technique. Subsequently, ensemble learning employs confidence averaging to integrate the enhanced models. TL-CNN-IDS exhibited accuracies of 99.85% and 99.53%, respectively, when evaluated on the CICIDS2017 and NSL-KDD datasets [15].

Nandanwar et al. presented a strong model for identifying and categorizing botnet attacks in IoT systems. This is achieved by combining a hybrid CNN-BiLSTM architecture with transfer learning (TL-BiLSTM). The model focuses on detecting and mitigating Mirai and BASHLITE attacks on nine IoT devices. It performs exceptionally well on the N_BaIoT dataset, obtaining an accuracy of 99.52% across 10 different categories and outperforming the most advanced techniques by a margin of 3.2% to 16.07%. This underscores the efficacy of the methodology in enhancing the detection of botnet attacks in IoT systems [16].

Using Federated Learning and Transfer Learning approaches in combination with several Machine Learning and Deep Learning algorithms to improve Intrusion Detection Systems, Otoum et al. investigated enhancing the security of IoT applications, with an eye towards the Internet of Medical Things (IoMT). Their research shows that compared to conventional approaches, FL and TL can improve model performance, accelerate the learning process, lower the data needed for training, and protect user privacy. With the Edge-IIoTset dataset, the suggested method achieves 99.3% average accuracy [17].

Rashid et al. presented a FL approach to detect undesired intrusions without violating data privacy, hence improving the security and privacy of IoT networks. The proposed FL method trains models locally on IoT devices, sharing just parameter changes with a central server, unlike conventional centralized machine learning methods that demand enormous volumes of centralized data. After aggregating the updates and improving the detection technique, this server distributes the devices with further training using the refined detection method. This distributed method maximizes the general model and preserves privacy. Extensive studies on a new dataset Edge-IIoTset show the method's effectiveness; its accuracy is 92.49%, quite similar to the 93.92% accuracy of traditional centralized ML models [18].

By suggesting an End-to-End learning, non-complex CNN1D model meant to detect advanced threats in I-IoT environments, Hassini et al. addressed the essential challenge of cybersecurity in the Industrial-Internet of Things (IIoT). With a minimum loss of 0.0011, the model detects all 15 classes in the Edge-IIoTset dataset — which comprises 14 complex attack types targeting real I-IoT networks — achieves a startling accuracy of 99.96% and K-fold cross-validation confirms the generalizability and resilience of the model to real-world I-IoT situations, hence validating its efficiency [19].

Govindaram et al. presented FL and Blockchain (BC) based Intrusion Detection System (FLBC-IDS) for Internet of Things environments combining EfficientNet, Hyperledger Blockchain, and Horizontal Federated Learning (HFL) to improve security. While Hyperledger Blockchain guarantees transparent and tamper-resistant record of model modifications, HFL enables safe, distributed model training across IoT devices. From network traffic data, EfficientNet enhances feature extracting and classification. FLBC-IDS performs highly on CICIDS-2018 and CICIoT-2023 datasets with 98.89% accuracy, 98.044% recall, 98.29% F1-score, and 98.44% precision. The research emphasizes the characteristics of the approach and its possibility for practical IoT security uses since it offers strong protection against cyberattacks [20].

Several studies have suggested innovative DTL methods for detecting intrusions in IoT networks. Nevertheless, these current systems have various constraints. Primarily, the majority of the offered techniques concentrate on identifying a restricted spectrum of assaults and depend on obsolete information for assessment. These datasets are not a precise reflection of the current IIoT landscape. In order to overcome this constraint, our research makes use of the most up-to-date IIoT cybersecurity dataset, Edge-IIoTset, which allows the proposed scheme to accurately categorize 14 distinct forms of cyberattacks.

Another notable limitation of current research is the insufficient characterization of devices. Our proposed system differs from others by using a cybersecurity dataset that accurately represents the behavior of 10 distinct IoT devices. This addition improves the integrity and dependability of the model. In addition, the majority of studies commonly employ a limited number of pre-trained models, which might have a detrimental effect on the accuracy and resilience of attack detection.

To address this issue, our suggested approach utilizes nine pre-trained CNN models and optimizes their hyper-parameters through the use of random search. Following the training process, we carefully choose three models from the top performing models that exhibited the highest performance. These models were then combined using the bootstrap aggregation (bagging) ensemble technique. This strategy greatly improves the strength and precision of the suggested intrusion detection framework.

3. Proposed scheme

This section outlines the research methodology employed in the proposed scheme. We begin by introducing the dataset, pre-processing the data and transformation into suitable format. Subsequently, we delve into the core components of the scheme: DTL for leveraging pre-trained models, hyper-parameter optimization for fine-tuning the models, and application of ensemble technique for potentially improving performance. Finally, a visual representation of the proposed scheme is provided through a block schematic Fig. 1.

3.1. Edge-IIoTset dataset

Our proposed scheme leverages the most recent, comprehensive, and reliable cybersecurity dataset for IoT and IIoT applications: the Edge-IIoTset. This publicly available dataset, meticulously prepared by Ferrag et al. [21], reflects a realistic IIoT environment. It encompasses data from over ten diverse IoT device types, including sensors for temperature, flame detection, humidity, soil moisture, water level, pH levels, and monitoring of heart rate. Additionally, the dataset incorporates data representing fourteen distinct threat categories relevant to IIoT and IIoT network protocols.

3.2. Data Pre-processing

Data pre-processing plays a critical role in training effective machine learning and deep learning models. The Edge-IIoTset dataset, initially containing 62 features categorized into 15 classes, underwent several pre-processing steps. First, we confirmed the absence of missing values (NaN). Second, a minimal number of duplicate rows (0.037% or 815 instances) were removed. Third, feature engineering involved eliminating irrelevant columns that minimally impact model predictions. These features included network protocol details (e.g., “arp.src.proto_ipv4”), payload data (e.g., “http.file_data”), and timestamps (e.g., “frame.time”). The resulting dataset comprised 1,909,671 rows and 47 features. Notably, one column contained eight categorical features. We employed one-hot encoding for all except the target variable (“Attack_type”). This category included attributes like “http.request.method” and “mqtt.topic”. The final pre-processed dataset comprises of 1,909,671 rows and 96 features and was exported as a CSV file for further analysis. The preprocessed dataset has been summarized in Table 1.

Table 1

Summary of the pre-processed Edge-IIoTset dataset.

Dataset	Entries	Features	Classes	Transformed images	Feature types
Train	1 527 737	96	15	5369	Numerical (47)
Test	381 934	96	15	1342	Categorical (49)
Total	1 909 671	96	15	6711	96

3.3. Data transformation

The proposed DTL scheme leverages pre-trained CNN architectures. While CNNs excel at image data, the preprocessed Edge-IIoTset is a structured tabular dataset. To address this, we transform the data into a format suitable for CNNs.

The initial step involves data normalization. We employ quantile normalization, which transforms the feature distribution into a normal one. This ensures all feature values fall within a specific range, typically 0 to 255 for image pixel values [22]. This effectively handles outliers, as most values tend to align with the median after normalization.

Following normalization, data samples are rearranged into image blocks based on the dataset’s feature size. The preprocessed Edge-IIoTset has 95 features. We convert each set of 285 consecutive samples (each with 95 features) into a color image with dimensions of $95 \times 95 \times 3$. These images are then classified based on attack patterns.

When all of the samples in an image show typical behavior, the image is classified as “Normal”. On the other hand, in the event that the image includes attack samples, it is classified as the most common sort of attack in that block. The Edge-IIoTset dataset includes labels for 14 attack categories, such as DDoS attacks (UDP, ICMP, TCP, HTTP), vulnerability scanner, password attacks, upload, backdoor, port scan, cross-site scripting (XSS), ransomware, fingerprinting, and man-in-the-middle (MITM) attacks. Fig. 2 showcases representative samples for each attack category. It illustrates representative sample images for each of the 14 distinct cyberattack categories identified within the Edge-IIoTset dataset. Each image has been transformed from the structured tabular data format into a visual representation suitable for CNN processing.

3.4. CNN-based deep transfer learning

CNNs are a powerful deep learning architecture widely used for image identification and categorization tasks [23]. Unlike other methods, CNNs can directly process images without requiring supplementary feature extraction or data reconstruction steps. A typical CNN architecture comprises three essential layers: convolutional, pooling, and fully connected.

- **Convolutional Layers:** These layers automatically extract features from images using the convolution operation.
- **Pooling Layers:** These layers reduce the complexity of the data while preserving crucial information through local correlations.
- **Fully Connected Layers:** The final layer consolidates all extracted features and produces the classification output.

DTL involves transferring knowledge gained from a pre-trained model on one dataset to a new, target dataset [24]. This technique has proven successful in various image processing and classification applications. Lower layers in CNN models typically capture general feature patterns applicable across diverse tasks. Conversely, features learned by the upper layers are particular to the training dataset. That being the case, leveraging the lower layers of pre-trained CNNs for new applications offers a significant advantage [24]. Tweaking further enhances the performance of DTL models. During fine-tuning, most pre-trained model layers are frozen, while the upper layers are unfrozen to adapt the model to the new dataset. This process allows the model to refine the higher-level features learned from the pre-trained model for better alignment with the specific target dataset [24].

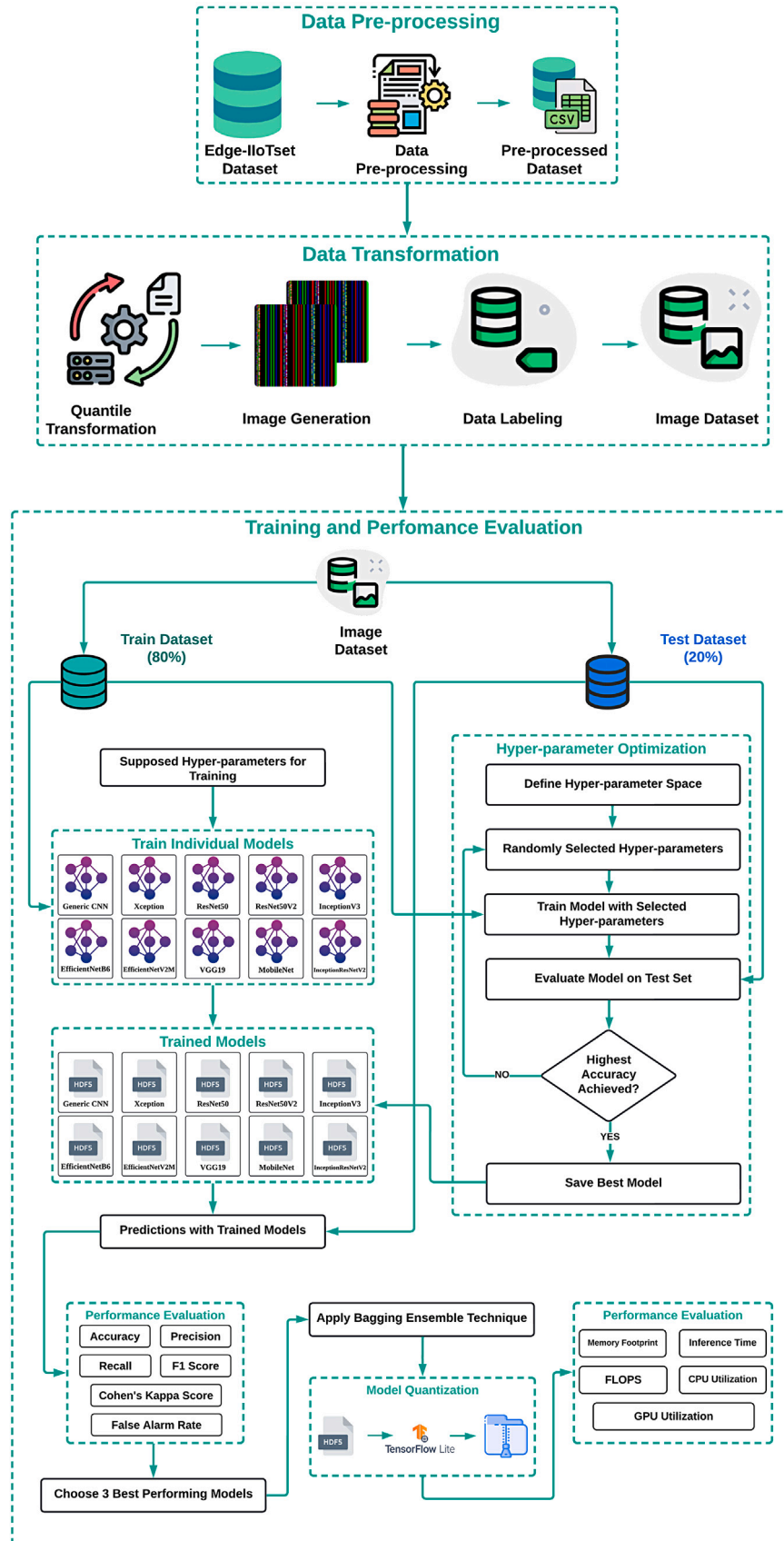


Fig. 1. Block diagram illustrating the overall working of the proposed methodology.

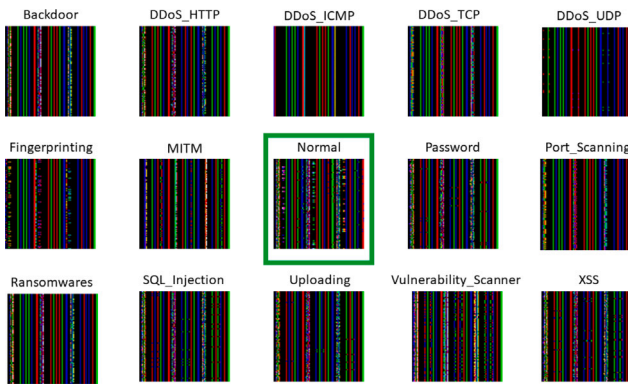


Fig. 2. Sample image from each of the 14 distinct cyberattack categories.

3.4.1. Xception

In 2017, Chollet introduced Xception, a sophisticated deep CNN structure that improves the inception model by utilizing depthwise separable convolutions [25]. The model's efficacy and performance are significantly improved by this approach. Xception has demonstrated exceptional performance on numerous image classification benchmarks, underscoring its ability to extract and acquire complex characteristics from input data. Xception reinvents the inception architecture with depthwise separable convolutions, resulting in computational efficiency and cutting-edge performance. This renders it a dependable solution for a diverse array of deep learning duties.

3.4.2. ResNet50

In 2016, He et al. introduced ResNet, a deep CNN design that resolves the vanishing gradient issue, a common training challenge for extremely deep neural networks [26]. This approach is effective, as evidenced by ResNet50, a specialized variant of the ResNet architecture that contains 50 convolutional layers. It has achieved cutting-edge results on a variety of image classification tasks, demonstrating that residual learning is a valuable approach to resolving the vanishing gradient problem.

3.4.3. ResNet50V2

ResNet50V2 enhances the efficiency and performance of ResNet50 by integrating new features. ResNet50V2 includes substantial improvements, including preactivation to improve training stability and bottleneck blocks to reduce the number of parameters. These advancements improve ResNet50's capabilities for image classification tasks.

3.4.4. InceptionV3

InceptionV3 enhances the efficacy and achieves superior results in picture categorization by enhancing the Inception architecture. By deconstructing convolutions (which decreases computing costs) and employing batch normalization (which improves gradient flow), this is achieved. This method allows InceptionV3 to attain high accuracy and processing efficiency [27].

3.4.5. EfficientNetB6

A novel approach known as "compound scaling" was introduced by Tan and Le in 2019 to improve the performance of CNN architectures that are expressly designed for image categorization. This approach is part of the EfficientNet family. By consistently adjusting the depth, width, and resolution, this method accomplishes a harmonious balance between resource utilization and performance. This strategy is illustrated by a specific version, EfficientNetB6, which achieves a high level of accuracy while utilizing fewer parameters and computations than conventional CNNs [28].

3.4.6. EfficientNetV2M

EfficientNetV2, an extension of EfficientNet [28], employs neural architecture search (NAS) to improve performance and efficiency. The network architecture is improved by the NAS approach, which leads to variants like EfficientNetV2M that accomplish a superior balance between resource efficiency and accuracy in comparison to previous versions [29].

3.4.7. VGG19

The VGG19 model, developed by Simonyan and Zisserman in 2014, places a higher emphasis on achieving accuracy rather than optimizing for efficiency. VGG19 achieves great picture classification performance despite its complex design consisting of 19 layers and its use of basic 3×3 filters [30]. Nevertheless, this advantage is counterbalanced by the substantial processing requirements and memory utilization, which are higher in comparison to contemporary models such as EfficientNet.

3.4.8. MobileNet

MobileNet, as described by Howard et al. in 2017, focuses on optimizing performance for mobile devices. The model utilizes depthwise separable convolutions, similar to Xception, and incorporates linear bottleneck layers to greatly decrease the number of parameters and computations in comparison to conventional CNNs [31]. MobileNet is able to attain high accuracy in classifying images on devices that have minimal computational capabilities.

3.4.9. InceptionResNetV2

InceptionResNetV2, proposed by Szegedy et al. (2016), integrates the advantages of Inception modules and residual connections derived from the ResNet architecture. This highly advanced model, with an impressive depth of 164 layers, demonstrates exceptional proficiency in acquiring intricate characteristics from extensive image collections [27]. This enables it to efficiently classify photos into 1000 discrete categories.

3.5. Optimizing the hyper-parameters

Regulating the CNN architectures' hyper-parameters is essential to get the best potential performance of the fundamental models with the provided dataset. Careful choice of these hyper-parameters can remarkably affect the model's efficiency and performance. The conventional perspective of discovering the best hyper-parameters for a certain dataset is a manual search, depending on the person's experience building deep learning models to solve related complications. Manual search has primarily the disadvantage of being able to determine the finest parameters for a given dataset, but these settings might not be of the highest quality for other datasets. Thus, manual choosing is not a suitable approach for hyper-parameter adjustment in many experimental circumstances. Additional automated and assisted methodologies are need to explore various configurations of deep learning models for many issues. The prevailing hyper-parameter optimization (HO) techniques for CNNs in current research include Random Search (RS), Grid Search (GS), gradient-based optimization (GO) and Bayesian Optimization (BO) [32].

RS is employed in this investigation to HO of CNN models. It is a straightforward approach that is in stark contrast to more complex optimization techniques. In this context, we establish a defined and permissible range for each hyper-parameter. The method proceeds by selecting random combinations of these parameters repeatedly, resulting in unique configurations of CNN models. In the subsequent phase, each configuration is evaluated in accordance with a performance metric, such as the accuracy of validation. The configuration that produces the maximum performance metric is referred to as the optimal hyper-parameter set.

RS eliminates the additional computational burden that results from the use of crossover and mutation operators, which are present in

specific optimization methods. Rather, it is contingent upon the arbitrary selection of a diverse array of hyper-parameter combinations for the purpose of exploration. This investigation allows the algorithm to effectively identify favorable regions within the search space and potentially prevent it from becoming ensnared in suboptimal solutions. RS is frequently chosen for HO, particularly when faced with extensive search areas or limited computer resources, due to its simplicity and efficacy [33]. Algorithm 1 outlines the comprehensive operation of RS Optimization.

Algorithm 1 Random Search for Hyper-parameter Optimization

Input: $dataset$ $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$,
Hyperparameter Space: H base learning algorithms: A ,
number of rounds: N

Process:

- 1: Initialize $BestParams$
- 2: Initialize $BestScore$
- 3: **for** $k = 1, 2, \dots, N$: **do**
- 4: $h_k = \text{SampleHyper-Parameters}(H)$ $\triangleright$ Randomly sample hyper-parameters from the hyper-parameter space
- 5: $model_k = L(D, h_k)$
- 6: $score_k = \text{EvaluateModel}(model_k)$
- 7: **if** $score_k > BestScore$ **then**
- 8: $BestScore = score_k$
- 9: $BestParams = h_k$
- 10: **end if**
- 11: **end for**

Return: $BestParams, BestScore$

3.6. Ensemble learning

Ensemble learning is an advanced method in machine learning that merges the predictions of several basic models to get a more precise and resilient model overall [34]. This strategy exploits the advantages of each unique model to alleviate their limitations, resulting in enhanced performance and generalization. Bagging, also known as Bootstrap Aggregation, is a technique introduced by Breiman et al. It includes training many iterations of a model on various subsets of the training data and then averaging their predictions. This process helps to minimize variance and prevent overfitting [35].

Ensemble learning has demonstrated significant efficacy in real-world applications. For example, the Random Forest algorithm, which is a sort of bagging method, has demonstrated greater performance in different fields when compared to individual models [36]. Ensemble learning techniques leverage the collective predictive ability of several models, resulting in more precise and dependable outcomes compared to individual models [37]. This article utilizes the bootstrap aggregation ensemble approach to integrate the results of three top-performing models, resulting in a more precise and resilient model. The process of bootstrap aggregation is thoroughly described in Algorithm 2.

Algorithm 2 Bootstrap Aggregation (Bagging)

Input: $dataset$ $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$,
base learning algorithms: A ,
number of rounds: T

Process:

- 1: **for** $t = 1, 2, \dots, T$: **do**
- 2: $D_t = \text{GenerateBootstrapSample}(D)$ $\triangleright$ Generate a bootstrap sample from dataset
- 3: $model_t = A(D_t)$ $\triangleright$ Trains a base learning model $model_t$ from the bootstrap sample D_t
- 4: **end for**

Output: $H(x) = \arg\max_{y \in Y} \sum_{t=1}^T l(y = model_t(x))$

3.7. Model quantization

Quantization of neural networks is an advanced technique used to compress models, thereby reducing their resource utilization while maintaining performance levels. This method involves converting the continuous values of a neural network's parameters and activations into discrete, lower-bit representations. By reducing the precision of the weights and activations, quantization significantly decreases the model size and computational requirements, making it more suitable for deployment in resource-constrained environments such as mobile devices and edge computing. Quantized models consume less memory and have faster inference times, leading to enhanced efficiency and reduced latency [38]. Additionally, quantization can mitigate the overfitting problem by introducing noise during training, which acts as a regularizer [39].

4. Implementation setup & performance metrics

This section presents the implementation setup of the proposed scheme and metrics used for performance evaluation.

4.1. Implementation setup

We put the suggested DTL architecture into action and evaluate its efficacy on the Google Colab Pro platform with the help of the Keras libraries. An Nvidia Turing T4 GPU, equipped with 16 GB of graphical processing unit (GPU) memory and 15 GB of random access memory (RAM), is available with Google Colab Pro, guaranteeing that deep learning algorithms will run smoothly. Using Keras's ImageDataGenerator function, the primary step is to retrieve the Train and Test image datasets from Google Drive. Then, ten CNN models that have already been trained are brought forward into the Keras environment. A callback function is used to store the best model according to validation accuracy during the training phase, keeping the number of epochs to 30. The models with highest performance are stored in Google Drive for later use post training, and their efficacy is evaluated with the help of the test dataset.

4.2. Performance evaluation metrics

Prior to delving into the precise details of performance evaluation metrics, it is important to develop a shared understanding by examining and clarifying some essential terms. This will guarantee that we all have a shared understanding when it comes to evaluating these metrics.

True Positive (TP): This pertains to a scenario in which the model accurately recognizes a positive state. For instance, an intrusion detection system accurately recognizing a cyberattack.

True Negative (TN): This refers to a scenario in which the model accurately detects a negative situation. For instance, an intrusion detection system accurately recognizing regular network traffic without any attack.

False Positive (FP): This pertains to a scenario in which the model erroneously classifies a negative state as positive. This is commonly referred to as a Type I mistake. As an illustration, an intrusion detection system erroneously identifies regular traffic as an assault, resulting in a false alarm.

False Negative (FN): This pertains to a scenario in which the model erroneously classifies a situation as negative when it is, in fact, positive. This is alternatively referred to as a Type II mistake. For instance, a scenario when an intrusion detection system is unable to identify a genuine cyberattack.

This terminology is summarized in Table 2.

Furthermore, the performance of the proposed scheme is evaluated using the following metrics.

Table 2
Performance comparison of different CNN models trained using non-optimized hyper-parameters.

Term	Description	Example
True Positive (TP)	Precisely identifies positive condition	Cyberattack identified correctly
True Negative (TN)	Precisely identifies negative condition	Normal traffic identified correctly
False Positive (FP)	Incorrectly identifies positive condition (Type I error)	Normal traffic flagged as an attack (false alarm)
False Negative (FN)	Incorrectly identifies negative condition (Type II error)	Cyberattack missed by the system

- Accuracy

Accuracy is a fundamental measure used to assess the effectiveness of a classification model. It quantifies the accuracy of the model by determining the percentage of true predictions.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

- Precision

Precision refers to the accuracy of the model's positive predictions. The question inquires about the number of accurate predictions among all the positive ones we made.

$$Precision = \frac{TP}{TP + FP} \quad (2)$$

- Recall

Recall is the process of identifying all the instances that are positive. The question inquires about the number of successfully predicted positive data points out of the total number of positive data points that should have been forecasted.

$$Recall = \frac{TP}{TP + FN} \quad (3)$$

- F1 Score

The F1 score quantifies the balance between precision and recall. The harmonic mean is used to calculate a single score that takes into account both measures.

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall} \quad (4)$$

- Cohen's Kappa Score (CKS)

The Cohen's Kappa score beyond mere accuracy by considering the agreement that occurs by chance. It is beneficial for assessing classification models, particularly when dealing with imbalanced datasets.

$$CKS = \frac{Observed\ Accuracy - Expected\ Accuracy}{1 - Expected\ Accuracy} \quad (5)$$

where

Observed Accuracy is the percentage of agreements among raters, when classifying items into categories.

Expected Accuracy is the level of agreement that could be achieved by chance alone.

- False Alarm Rate (FAR)

The false alarm rate, also known as the false positive rate (FPR), is a metric that measures the frequency at which a model incorrectly classifies negative instances as positive. Specifically, it quantifies the proportion of actual negative instances that are incorrectly identified as positive by the model.

$$FAR = \frac{FP}{FP + TN} \quad (6)$$

- Memory footprint

The memory footprint is a metric that quantifies the amount of memory that a machine learning model consumes during its execution. It is a critical factor to consider, especially when implementing models on devices with limited memory resources.

- Inference time

The duration required for a trained machine learning model to analyze a single input and produce a corresponding output is

referred to as inference time. This duration is typically measured in milliseconds (ms) or seconds (s) and is indicative of the computational expense associated with the model's predictions.

- Floating Point Operations Per Second (FLOPS)

FLOPS is a metric that is used to evaluate the computational efficiency of a system or model by determining the number of floating-point operations that can be executed within a single second. FLOPS can be implemented to assess the computing efficacy of an intrusion detection model during its operations.

- CPU Utilization

Central processing unit (CPU) utilization is the percentage of processing power that the central processing unit (CPU) is utilizing at any given instant. CPUs are essential for the execution of various calculations in machine learning tasks. To evaluate the efficacy of the IDS paradigm in utilizing the CPU's resources, it is crucial to monitor CPU consumption.

- GPU Utilization

This pertains to the degree to which the graphics processing unit (GPU) is utilized during the operation of the intrusion detection model. GPUs are frequently employed in machine learning projects to expedite computations. Tracking GPU consumption provides insights into the efficiency with which the IDS model employs GPU resources, hence affecting the way it performs.

5. Experimental results

The experimentation and performance analysis were carried out in two separate steps. During the initial step, a set of hyper-parameters was defined for each base learning model, and these models were trained utilizing those defined hyper-parameters. During the second step, we utilize a RS algorithm to ascertain the most favorable hyper-parameters. Afterwards, all fundamental learning models were trained utilizing these optimal hyper-parameters. In order to maintain uniformity in the outcomes, the number of epochs was set at 30 for both scenarios. We kept saving the best performing base model at each epoch on the basis of validation accuracy. The top-performing base learning models, chosen by the highest validation accuracy. We kept the trained base learning models in Google Drive for future utilization in predictions.

5.1. Assessment in non-optimized conditions

In order to assess the predictive accuracy of each model under non-optimized conditions, we initially imported all the trained models and test datasets from Google Drive. Subsequently, we acquired the forecast outcomes for each model, evaluating their performance based on accuracy, precision, recall, false alarm rate, F1 score, Cohen's Kappa score, and the confusion matrix. The condensed forecast outcomes of all models without HO are displayed in Table 3. The Xception model demonstrated the best level of accuracy in identifying attacks, achieving an accuracy of 96.94%. The model that performed second best was InceptionV3, achieving an accuracy of 96.27%. It was followed by the generic CNN model, which had an accuracy of 95.9%. The subsequent stage entailed incorporating the results of the most successful models. According to their highest levels of accuracy in detecting attacks, we have chosen the three most effective models: Generic CNN, Xception, and InceptionV3.

Table 3

Performance comparison of different CNN models trained using non-optimized hyper-parameters.

Model name	Accuracy	Precision	Recall	FAR	F1 score	CKS
Generic CNN	0.95901	0.95163	0.95901	0.00205	0.94904	0.91783
Xception	0.96944	0.95838	0.96944	0.00204	0.96285	0.93877
ResNet50	0.93815	0.91600	0.93815	0.00414	0.92354	0.87614
ResNet50V2	0.87332	0.86448	0.87332	0.00851	0.85077	0.73933
InceptionV3	0.96274	0.93834	0.96274	0.00249	0.94835	0.92520
EfficientNetB6	0.76825	0.71579	0.76825	0.01569	0.72548	0.52614
EfficientNetV2M	0.94336	0.92042	0.94336	0.00378	0.92928	0.88650
VGG19	0.93964	0.93032	0.93964	0.00404	0.92307	0.87895
MobileNet	0.90163	0.85955	0.90163	0.00660	0.87246	0.80253
InceptionResNetV2	0.87853	0.83051	0.87853	0.00816	0.84717	0.75598
Bagging	0.98062	0.96993	0.98062	0.00414	0.97379	0.96117

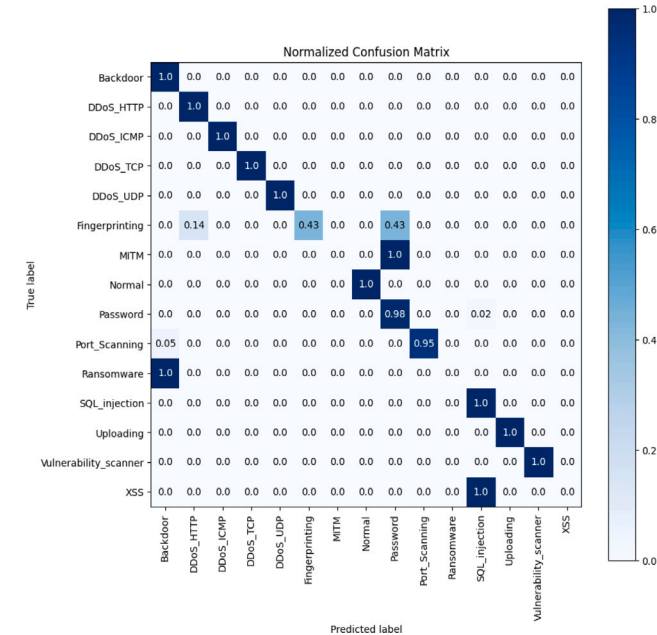


Fig. 3. Accuracy performance of various non-optimized CNN models and the ensemble approach used in the proposed IDS for IoT and IIoT networks.

We enhance the accuracy of proposed scheme using the Bootstrap Aggregation (bagging) Ensemble method. In particular, this approach effectively reduces the prediction model's variance while handling variance trade-offs, therefore improving the model's accuracy. In terms of attack detection, the final ensemble model achieved a precision of 98.06%. Furthermore, the last IDS showcased outstanding performance by way of the following metrics: recall of 98.06%, F1 score of 97.38%, and Cohen's kappa score of 96.11%. Fig. 3 displays the conclusive ensemble model's confusion matrix in the non-optimized configuration.

5.2. Assessment in optimized conditions

We utilized the RS approach to discover the most effective hyper-parameters for our foundational learning models. This strategy is anticipated to produce better results in comparison to models trained using non-optimized, randomly selected hyper-parameters. The train accuracy, test accuracy, train loss and test loss for optimal setting for all models are shown in Fig. 4. After optimization, the trained models were saved in Google Drive for future utilization in prediction tasks.

In order to thoroughly evaluate the predicted precision of each model under the optimal circumstances, we obtained all the trained models and their accompanying test datasets from Google Drive. Afterwards, we produced predictions for each model and assessed their effectiveness using a range of metrics: accuracy, precision, recall, false

Table 4

Performance comparison of different CNN models trained using optimized hyper-parameters.

Model name	Accuracy	Precision	Recall	FAR	F1 score	CKS
Generic CNN	0.99925	0.99929	0.99925	0.0	0.99923	0.99850
Xception	0.99478	0.99108	0.99478	0.00034	0.99262	0.98955
ResNet50	0.98658	0.98792	0.98658	0.00089	0.98519	0.97312
ResNet50V2	0.99254	0.99345	0.99254	0.00049	0.99176	0.98507
InceptionV3	0.99850	0.99866	0.99850	0.0	0.99842	0.99701
EfficientNetB6	0.99403	0.99036	0.99403	0.00039	0.99187	0.98805
EfficientNetV2M	0.99254	0.99303	0.99254	0.00049	0.99231	0.98507
VGG19	0.98807	0.98923	0.98807	0.00079	0.98767	0.97611
MobileNet	0.99701	0.99701	0.99701	0.00019	0.99701	0.99403
InceptionResNetV2	0.99478	0.99551	0.99478	0.00034	0.99489	0.98955
Bagging	1.0	1.0	1.0	0.0	1.0	1.0

alarm rate, F1 score, Cohen's Kappa score, and the confusion matrix. Table 4 summarizes the prediction results of all models using the improved hyper-parameters. The generic CNN model, which is not specific to any one application, exhibited the highest level of performance, attaining an accuracy rate of 99.92%. The model achieved a 100% accuracy in successfully classifying 13 different classes, with only 14% of samples in one particular class being misclassified. The InceptionV3 model, which ranked second in performance, achieved a remarkable attack detection accuracy of 99.85%. Additionally, it successfully identified 13 classes of the Edge-IIoTset dataset. However, it misclassified 29% of the samples in one particular class. The MobileNet model, the third model in the series, attained an impressive accuracy rate of 99.7% and successfully classified 12 different classes with precision. The performance of these optimized models was compared with the non-optimized versions, as depicted in Fig. 6.

In order to improve performance, the results of the high-performing models, namely Generic CNN, InceptionV3, and MobileNet, were combined using the Bootstrap Aggregation (bagging) Ensemble technique. The ultimate ensemble model attained a 100% accuracy in detecting attacks. In addition, the performance measures for the final IDS, such as precision, recall, false alarm rate, F1 score, and Cohen's kappa (CK) score, all achieved a perfect score of 100%. The proposed approach accurately identified 14 classes with a 100% success rate. The confusion matrix for the optimized model is displayed in Fig. 5. After evaluation results, we find that the performance of generic CNN, InceptionV3 and MobileNet is best. Moreover, the size of InceptionV3 and MobileNet models is 85 MB and 13 MB, respectively. Combining their output through ensemble technique would help us towards our goal, i.e. lightweight intrusion detection scheme. Fig. 7 illustrates the performance comparison between the highest-performing individual models and the ultimate ensemble model.

5.3. Comparison with existing frameworks

We compare the performance of our proposed scheme with the existing schemes which we have covered in the literature review, Section 2. The researchers have mostly evaluated their proposed frameworks using the old datasets. These datasets have limited classes and can identify attacks related to those classes only. Researchers have claimed an accuracy of 99.85% and 99.53% using the datasets CI-CIDS2017 and NSL-KDD, respectively [15]. Their proposed transfer learning based scheme only detects seven classes of attacks accurately. We use a comprehensive and latest cybersecurity dataset which contains both IoT and IIoT data. This dataset 'Edge-IIoTset' has 15 classes and 14 types of attacks. We also use RS algorithm for finding optimal hyper-parameters, which optimizes the scheme's performance. We further apply quantization to reduce the size and hence making it a lightweight solution without compromising on accuracy. The comparison is illustrated in Table 5.

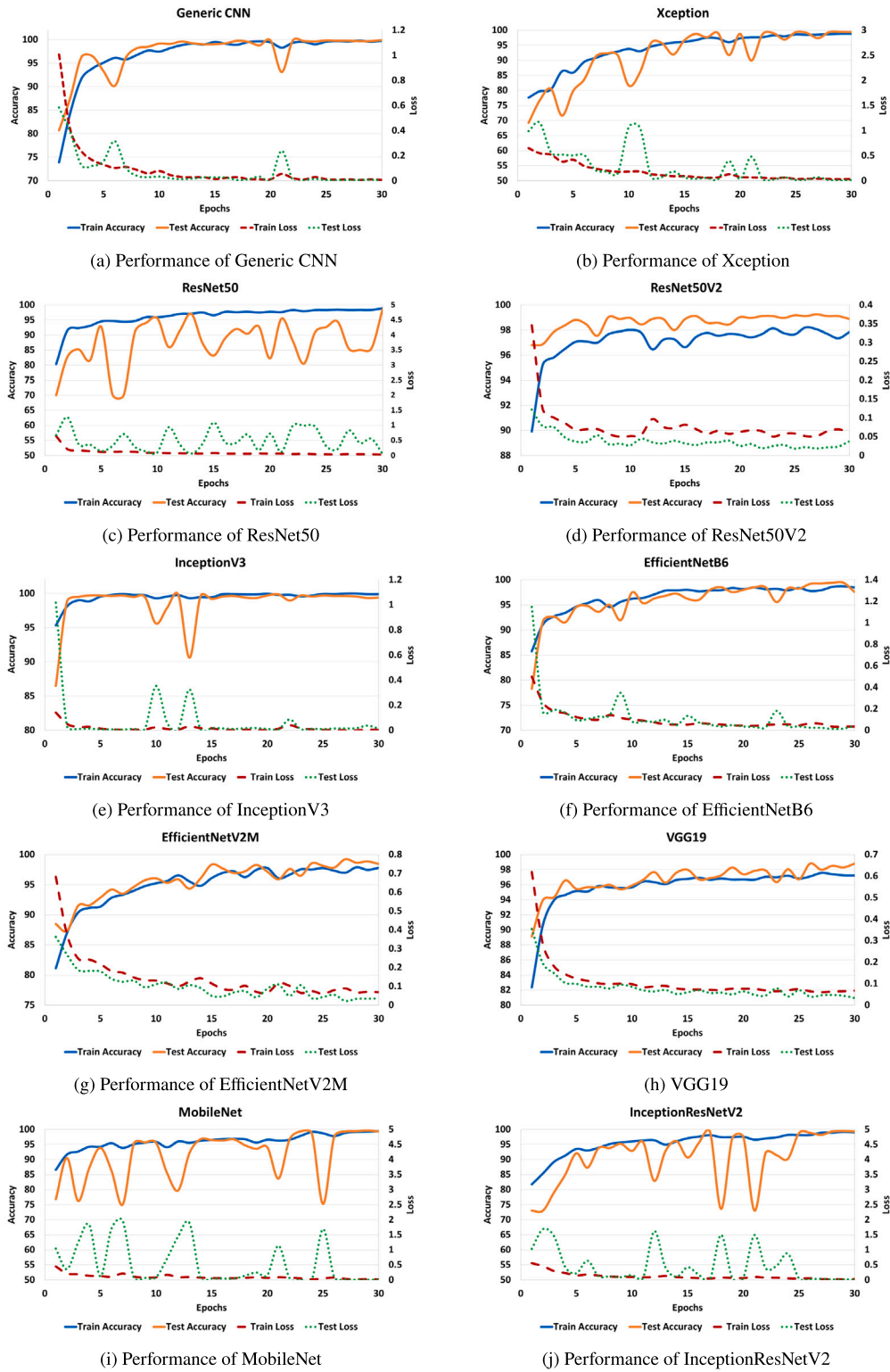


Fig. 4. Comparison of train and test accuracies and losses of various optimized CNN models.

5.4. Application of quantization

TensorFlow Lite provides a conversion procedure that allows for the transformation of a trained Keras model into a more lightweight and efficient structure. This is particularly useful when deploying machine

learning models on devices with restricted resources. The initial conversion stage makes use of the default capabilities of the TensorFlow Lite Converter, which produces a standard TFLite model that maintains the fundamental functionality of the original Keras model. Additionally, in order to optimize the model for use in contexts with limited resources,

Table 5
Performance based comparison of proposed scheme with other related IDSs in literature.

Framework	Year	Used method	Dataset	HO	Accuracy
Gou et al. [9]	2009	Distributed TL	KDD Cup 99	No	97.3%
TL-NID [7]	2020	TL	NSL-KDD	Yes	89.30%
Li et al. [5]	2021	TL	AWID	No	92%
Xu et al. [11]	2021	TL	CTU-13	No	93.01%
Singh et al. [12]	2021	DTL	ISCXVPN2016	No	96%
Mehedi et al. [8]	2021	DTL	Self Generated	Yes	98.10%
Rodriguez et al. [14]	2022	TL	UNSW-NB15	No	97.89%
Mehedi et al. [6]	2022	DTL	ToN_IoT	No	87%
FT-CID [13]	2023	TL	RPL-IIoT	No	85.52%
FTL-CNN [17]	2022	FL,TL	Edge_IIoTset	No	99.3%
Bierbrauer et al. [10]	2023	TL	UNSW-NB15, CICIDS2017	No	96.89%
TL-CNN-IDS [15]	2023	TL	CICIDS2017, NSL-KDD	Yes	99.85%, 99.53%
Rashid et al. [18]	2023	FL	Edge_IIoTset	No	92.49%
CNN1D [19]	2024	E2E Learning	Edge_IIoTset	No	99.96%
TL-BILSTM [16]	2024	TL	N_BallIoT	No	99.52%
FLBC-IDS [20]	2024	FL,BC	CIC-IDS2017,CIC-IoT2023	No	98.89%
Proposed scheme	2024	DTL	Edge_IIoTset	Yes	*100%

* Represents statistical significance over state-of-the-art scores under the paired t-test ($p < 0.05$).

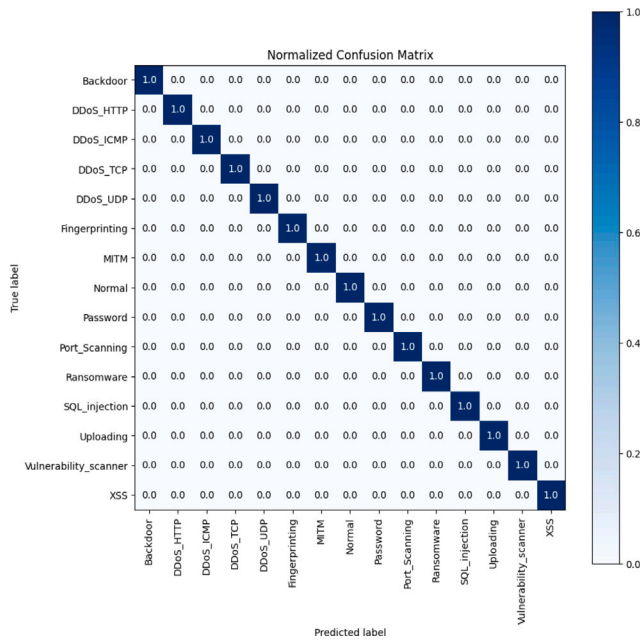


Fig. 5. Confusion matrix of ensemble accuracy for three best performing optimized models.

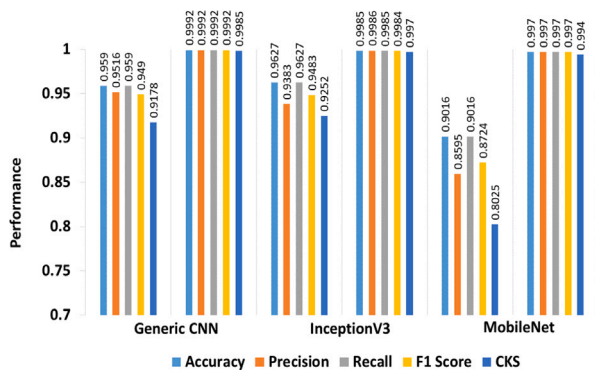


Fig. 6. Bar graph of performance comparison between top three performing models before and after optimization.

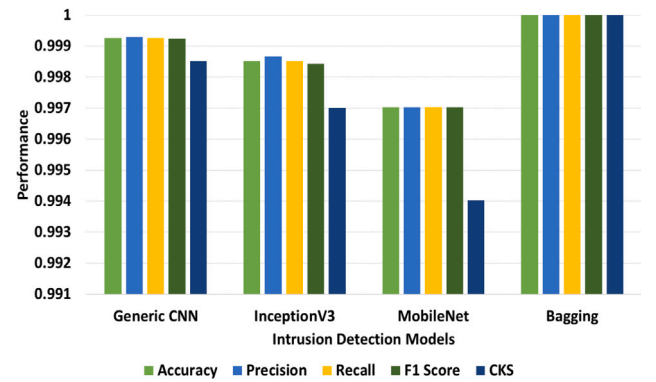


Fig. 7. Bar graph of performance comparison of the three best performing individual models and their combined ensemble model.

a post-training quantization strategy can be utilized. This method decreases the size of the model and enhances the speed of inference by using data types with lesser accuracy to describe the model's weights and activations. This streamlined and quantized version provides a compressed representation that is specifically designed for deployment on edge devices or systems with limited computational capabilities. Importantly, this procedure of quantization achieves improved efficiency without causing substantial compromises in the overall performance of the model.

To further explore the resource utilization, we apply quantization to the selected models. For this purpose, we convert the trained models in H5 format into TFLite format. Further, we apply default optimization and quantize the models. This further compresses the models' size. A comparison of memory footprints is shown in Table 6. Now we take these quantized models for further evaluation. We evaluate them for inference time, CPU utilization, GPU utilization and computational performance. The evaluation is listed in Table 7.

The computational complexity of the models used in this study is summarized in Table 7. The complexity is described using FLOPS, which is a standard measure for neural network models. Additionally, we report the hardware utilization, including GPU memory usage, to give a more complete picture of resource requirements.

We take the best three performing models, i.e., Generic CNN, InceptionV3 and MobileNet. We convert them to TFLite using the TensorFlow converter. and then apply quantization using default optimizations. The quantized models are smaller in size. We then combine their output through bagging ensemble technique to get a lightweight IDS. Performance evaluation for the individual quantized models and ensemble model is presented in Table 8. The results reveal that the

Table 6

Comparison of memory footprints of non-quantized and quantized CNN models.

Model name	Float model size (MB)	Quantized model size (MB)	Compression ratio
Generic CNN	1.137	0.294	3.869
Xception	80.328	20.780	3.865
ResNet50	91.626	23.315	3.929
ResNet50V2	90.743	23.187	3.913
InceptionV3	85.105	21.564	3.946
EfficientNetB6	157.454	41.900	3.757
EfficientNetV2M	202.587	53.894	3.758
VGG19	537.655	135.251	3.975
MobileNet	12.715	3.352	3.793
InceptionResNetV2	207.947	53.189	3.909

Table 7

Comparison of resource utilization and computational performance of different quantized CNN models trained on Edge_IIoTset dataset.

Model name	Inference time (ms)	CPU utilization (%)	GPU utilization (%)	Computational Performance (GFLOPS)
Generic CNN	7.904	54.4	0.774	9.426
Xception	7.328	63.3	7.337	7.290
ResNet50	3.600	53.2	7.337	7.733
ResNet50V2	4.653	54.3	4.003	7.954
InceptionV3	3.486	60.2	7.337	5.799
EfficientNetB6	5.783	61.8	14.003	6.734
EfficientNetV2M	4.291	60.6	14.003	10.731
VGG19	11.849	62.5	4.003	39.023
MobileNet	2.151	48.1	2.337	1.136
InceptionResNetV2	4.443	51.9	14.003	13.109

Table 8

Performance evaluation of three best-performing individual quantized models and their combined quantized model through ensemble learning.

Model name	Accuracy	Precision	Recall	FAR	F1 score	CKS
Generic CNN-Q	0.9985	0.99850	0.99850	0.0	0.99850	0.99701
InceptionV3-Q	0.99776	0.99784	0.99776	0.00017	0.99773	0.99552
MobileNet-Q	0.99701	0.99701	0.99701	0.00019	0.99701	0.99403
Bagging-Q	0.99925	0.99907	0.9993	0.0	0.99873	0.99849

performance remains nearly the same when using lightweight models, without compromising the accuracy.

6. Evaluation on car hacking dataset

To further demonstrate the generalization capabilities of the proposed scheme, we conducted tests on the Car Hacking Dataset, a real-world dataset widely used in the field of automotive cybersecurity. This dataset provides a diverse set of features and attack types, allowing us to rigorously evaluate the robustness and effectiveness of our approach in detecting and classifying cyberattacks within vehicular networks.

The Car Hacking Dataset consists of normal and attack data. It has data related to four types of attacks: Denial of Service (DoS) attack, fuzzy attack, gear spoofing, and RPM spoofing [23]. These characteristics make it an ideal candidate for testing the generalizability of our scheme. We perform preprocessing and data transformation to images on this dataset, similar to the Edge_IIoTset dataset in Section 3. After transformation of the dataset into images, it has 23383 images in training dataset and 5845 images in test dataset. The images belong to 5 classes including normal data.

We perform model training and evaluation on the Google Colab Pro platform with the help of the Keras libraries. A Google Tensor Processing Unit (TPU-v2), equipped with 64 GB of high bandwidth memory (HBM), is available with Google Colab Pro, guaranteeing that

Table 9

Comparison of resource utilization and computational performance of different quantized CNN models trained on car hacking dataset.

Model name	Inference time (ms)	CPU utilization (%)	GPU utilization (%)	Computational Performance (GFLOPS)
Generic CNN	0.930	50.7	0.879	15.658
Xception	1.533	18.1	7.337	9.793
ResNet50	0.691	27.4	7.337	11.903
ResNet50V2	1.096	39.8	4.004	12.054
InceptionV3	0.545	44.3	7.337	7.642
EfficientNetB6	1.777	46.9	14.004	8.839
EfficientNetV2M	0.704	63.4	14.004	17.321
VGG19	1.957	66.3	4.004	47.431
MobileNet	0.696	42.3	2.336	4.616
InceptionResNetV2	1.273	54.7	14.004	18.835

deep learning algorithms will run smoothly. We present the results of our evaluation on car hacking dataset, highlighting the performance of the proposed method in accurately detecting and classifying different types of attacks. Our scheme performs well and gives 100% results for accuracy, precision, recall, false alarm rate, F1 score and Cohen's kappa score and zero false alarm rate. Since the individual models are perfectly accurate in detecting attacks, we do not need ensemble learning to further enhance the performance.

We apply quantization to the models, trained on the car hacking dataset, and the memory footprint results have a negligible difference compared to the previous dataset. So we assume the results of memory footprints are the same as evaluated and presented in Table 6. The comparative analysis of resource utilization and computational performance of CNN models, trained on car hacking dataset, is presented in Table 9.

7. Conclusion

We present a streamlined and effective methodology for detecting unauthorized access in IoT and IIoT networks. This system combines DTL, HO, quantization and ensemble learning techniques to improve detection accuracy and efficiency. Furthermore, it utilizes quantization to attain a more efficient and less resource-intensive solution. The methodology we employ involves the utilization of nine sophisticated CNN designs, namely Xception, ResNet50, ResNet50V2, InceptionV3, EfficientNetB6, EfficientNetV2M, VGG19, MobileNet, and Inception-ResnetV2. The architectures are trained using the Edge-IIoTset dataset, which is an exhaustive and current cybersecurity dataset that includes both IoT and IIoT data. In order to optimize hyper-parameters, we utilize the RS technique, guaranteeing the optimal configuration for each model. The intrusion detection model is generated by integrating the results of the three best-performing models using a bagging ensemble technique. This approach improves the model's resilience and precision. We assess the efficacy of the proposed IDS scheme using diverse performance measures, showcasing its superiority over other cutting-edge IDSs in terms of accuracy in detecting attacks. In addition, we enhance our framework by using quantization on the models, resulting in a large reduction in their size without significantly affecting their accuracy. The accuracy of the proposed model after quantization exhibits a minimal disparity compared to its performance before the quantization, hence achieving a highly precise and efficient outcome.

CRedit authorship contribution statement

Basharat Ahmad: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Conceptualization. **Zhaoliang Wu:** Writing – review & editing. **Yongfeng Huang:** Supervision, Resources. **Sadaqat Ur Rehman:** Writing – review & editing.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Yongfeng Huang reports financial support was provided by National Natural Science Foundation of China. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was supported by the National Natural Science Foundation of China under grant number U2336208.

Data availability

Data will be made available on request.

References

- [1] M.A. Khan, K. Salah, IoT security: Review, blockchain solutions, and open challenges, *Future Generation Computer Systems* 82 (2018) 395–411.
- [2] S. Li, L.D. Xu, S. Zhao, The internet of things: a survey, *Information Systems Frontiers* 17 (2015) 243–259.
- [3] A. Sari, A. Lekidis, I. Butun, Industrial networks and IIoT: Now and future trends, *Industrial IoT: Challenges, Design Principles, Applications, and Security* (2020) 3–55.
- [4] K. Zhou, T. Liu, L. Zhou, Industry 4.0: Towards future industrial opportunities and challenges, in: 2015 12th International Conference on Fuzzy Systems and Knowledge Discovery, FSKD, IEEE, 2015, pp. 2147–2152.
- [5] X. Li, Z. Hu, M. Xu, Y. Wang, J. Ma, Transfer learning based intrusion detection scheme for internet of vehicles, *Inform. Sci.* 547 (2021) 119–135.
- [6] S.T. Mehedi, A. Anwar, Z. Rahman, K. Ahmed, R. Islam, Dependable intrusion detection system for IoT: A deep transfer learning based approach, *IEEE Trans. Ind. Inform.* 19 (1) (2022) 1006–1017.
- [7] M. Masum, H. Shahriar, Tl-nid: Deep neural network with transfer learning for network intrusion detection, in: 2020 15th International Conference for Internet Technology and Secured Transactions, ICITST, IEEE, 2020, pp. 1–7.
- [8] S.T. Mehedi, A. Anwar, Z. Rahman, K. Ahmed, Deep transfer learning based intrusion detection system for electric vehicular networks, *Sensors* 21 (14) (2021) 4736.
- [9] S. Gou, Y. Wang, L. Jiao, J. Feng, Y. Yao, Distributed transfer network learning based intrusion detection, in: 2009 IEEE International Symposium on Parallel and Distributed Processing with Applications, IEEE, 2009, pp. 511–515.
- [10] D.A. Bierbrauer, M.J. De Lucia, K. Reddy, P. Maxwell, N.D. Bastian, Transfer learning for raw network traffic detection, *Expert Syst. Appl.* 211 (2023) 118641.
- [11] M. Xu, X. Li, Y. Wang, B. Luo, J. Guo, Privacy-preserving multisource transfer learning in intrusion detection system, *Trans. Emerg. Telecommun. Technol.* 32 (5) (2021) e3957.
- [12] D. Singh, A. Shukla, M. Sajwan, Deep transfer learning framework for the identification of malicious activities to combat cyberattack, *Future Gener. Comput. Syst.* 125 (2021) 687–697.
- [13] N. Abosata, S. Al-Rubaye, G. Inalhan, Customised intrusion detection for an industrial IoT heterogeneous network based on machine learning algorithms called FTL-CID, *Sensors* 23 (1) (2022) 321.
- [14] E. Rodríguez, P. Valls, B. Otero, J.J. Costa, J. Verdú, M.A. Pajuelo, R. Canal, Transfer-learning-based intrusion detection framework in IoT networks, *Sensors* 22 (15) (2022) 5621.
- [15] F. Yan, G. Zhang, D. Zhang, X. Sun, B. Hou, N. Yu, TL-CNN-IDS: transfer learning-based intrusion detection system using convolutional neural network, *J. Supercomput.* 79 (15) (2023) 17562–17584.
- [16] H. Nandanwar, R. Katarya, TL-BILSTM IoT: transfer learning model for prediction of intrusion detection system in IoT environment, *Int. J. Inf. Secur.* 23 (2) (2024) 1251–1277.
- [17] Y. Otoum, V. Chamola, A. Nayak, Federated and transfer learning-empowered intrusion detection for IoT applications, *IEEE Internet Things Mag.* 5 (3) (2022) 50–54.
- [18] M.M. Rashid, S.U. Khan, F. Eusufzai, M.A. Redwan, S.R. Sabuj, M. Elsharief, A federated learning-based approach for improving intrusion detection in industrial internet of things networks, *Network* 3 (1) (2023) 158–179.
- [19] K. Hassini, S. Khalis, O. Habibi, M. Chemmakha, M. Lazaar, An end-to-end learning approach for enhancing intrusion detection in industrial-internet of things, *Knowl.-Based Syst.* 294 (2024) 111785.
- [20] A. Govindaram, et al., FLBC-IDS: a federated learning and blockchain-based intrusion detection system for secure IoT environments, *Multimedia Tools Appl.* (2024) 1–23.
- [21] M.A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, H. Janicke, Edge-IIoTset: A new comprehensive realistic cyber security dataset of IIoT and IIoT applications for centralized and federated learning, *IEEE Access* 10 (2022) 40281–40306.
- [22] S.-F. Lokman, A.T. Othman, M.H.A. Bakar, S. Musa, The impact of different feature scaling methods on intrusion detection for in-vehicle controller area network (CAN), in: *Advances in Cyber Security: First International Conference, ACeS 2019, Penang, Malaysia, July 30–August 1, 2019, Revised Selected Papers* 1, Springer, 2020, pp. 195–205.
- [23] H.M. Song, J. Woo, H.K. Kim, In-vehicle network intrusion detection using deep convolutional neural network, *Veh. Commun.* 21 (2020) 100198.
- [24] S. Shao, S. McAleer, R. Yan, P. Baldi, Highly accurate machine fault diagnosis using deep transfer learning, *IEEE Trans. Ind. Inform.* 15 (4) (2018) 2446–2455.
- [25] F. Chollet, Xception: Deep learning with depthwise separable convolutions, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 1251–1258.
- [26] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [27] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, Z. Wojna, Rethinking the inception architecture for computer vision, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 2818–2826.
- [28] M. Tan, Q. Le, Efficientnet: Rethinking model scaling for convolutional neural networks, in: *International Conference on Machine Learning*, PMLR, 2019, pp. 6105–6114.
- [29] M. Tan, Q. Le, Efficientnetv2: Smaller models and faster training, in: *International Conference on Machine Learning*, PMLR, 2021, pp. 10096–10106.
- [30] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, 2014, arXiv preprint [arXiv:1409.1556](https://arxiv.org/abs/1409.1556).
- [31] A.G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, H. Adam, Mobilenets: Efficient convolutional neural networks for mobile vision applications, 2017, arXiv preprint [arXiv:1704.04861](https://arxiv.org/abs/1704.04861).
- [32] T. Yu, H. Zhu, Hyper-parameter optimization: A review of algorithms and applications, 2020, arXiv preprint [arXiv:2003.05689](https://arxiv.org/abs/2003.05689).
- [33] J. Bergstra, Y. Bengio, Random search for hyper-parameter optimization., *J. Mach. Learn. Res.* 13 (2) (2012).
- [34] M. Al-Sarem, F. Saeed, A. Alsaedi, W. Boulila, T. Al-Hadhrani, Ensemble methods for instance-based arabic language authorship attribution, *IEEE Access* 8 (2020) 17331–17345.
- [35] L. Breiman, Bagging predictors, *Mach. Learn.* 24 (1996) 123–140.
- [36] T.G. Dietterich, Ensemble methods in machine learning, in: *International Workshop on Multiple Classifier Systems*, Springer, 2000, pp. 1–15.
- [37] G. Seni, J. Elder, *Ensemble Methods in Data Mining: Improving Accuracy Through Combining Predictions*, Morgan & Claypool Publishers, 2010.
- [38] R. Krishnamoorthi, Quantizing deep convolutional networks for efficient inference: A whitepaper, 2018, arXiv preprint [arXiv:1806.08342](https://arxiv.org/abs/1806.08342).
- [39] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, Y. Bengio, Quantized neural networks: Training neural networks with low precision weights and activations, *J. Mach. Learn. Res.* 18 (187) (2018) 1–30.



Basharat Ahmad received his BS degree in Computer Engineering from University of Engineering & Technology, Peshawar in 2012 and MS degree in Electrical Engineering from NUST School of Electrical Engineering and Computer Science (SECS), Islamabad, Pakistan in 2016. Currently, he is working as Research Assistant in the Tsinghua National Laboratory for Information Science and Technology, Tsinghua University, Beijing, China. His research interests include Computer Networks, Future Internet, Cloud Computing and Network Security.



Zhaoliang Wu received the PhD degree from the Communication University of China, Beijing, China, in 2024. He is a postdoctoral fellow at the Department of Electronic Engineering at Tsinghua University, with a research focus on machine learning, sentiment analysis, and data mining.



Yongfeng Huang is a full professor in the Department of Electronic Engineering, Tsinghua University, Beijing, China. Prof. Huang received the Ph.D. degree in CSE from Huazhong University of Science and Technology in 2000. In 2002 he joined Tsinghua University as a faculty after post-doctoral work in NGN Lab of Tsinghua University. He is an active researcher in Information System Theory and Engineering. Prof. Huang has published 200+ papers in Top Journals and International Conferences, including SCIENCE, IEEE Transactions on Knowledge and Data Engineering, AAAI, ACL et al.



Sadaqat Ur Rehman received his B.Sc. degree from the Department of Computer Systems Engineering, University of Engineering and Technology Peshawar and M.Sc. degrees from the Department of Electrical Engineering, Sarhad University of Science and IT in 2011 and 2014, respectively. Dr. Sadaqat ur Rehman pursued his Ph.D. degree with the Tsinghua National Laboratory for Information Science and Technology, Tsinghua University, Beijing, China. He worked as Artificial Intelligence Engineer in Schlumberger (Beijing Geoscience Centre), where he developed different Deep Learning/Machine Learning models for Drilling Dynamics Computation Engine. Currently, he is working as Assistant Professor in the School of Science, Engineering and Environment, University of Salford, United Kingdom.